

fhir-k8s-gitops: step-by-step guide

A tiny Node API that wraps the public HAPI FHIR test server, built into a Docker image by GitHub Actions, stored in GHCR, and deployed to Kubernetes (KillerCoda) by ArgoCD from Git.

Flow: code in Git → GitHub Actions builds image → GHCR stores it → YAML in Git names the image → ArgoCD makes the cluster match Git.

Repo: `github.com/rprrr736/fhir-k8s-gitops` (public) · **Image:** `ghcr.io/rprrr736/fhir-k8s-gitops:<tag>`
 · **Laptop folder:** `C:\Ravi\Ravi\AI learning\fhir-k8s-gitops`

Commands marked **PowerShell** run on your laptop. Commands marked **KillerCoda** run in the playground terminal.

Phase A: Local app

1. Create the project folder and check tools

Makes the working folder and confirms Node, npm and Git exist. Docker is not needed on the laptop, because GitHub builds the image.

```
cd "C:\Ravi\Ravi\AI learning"
mkdir fhir-k8s-gitops
cd fhir-k8s-gitops
node -v
npm -v
git --version
```

2. Initialize Node and install Express

Creates `package.json` and installs the web framework.

```
npm init -y
npm install express
```

3. Write the first version of the API

Two endpoints: `/health` (am I alive?) and `/hello` (does my app answer?). Create the file with `code` `index.js`, paste, save with `Ctrl+S`.

```
const express = require("express");
const app = express();
const PORT = process.env.PORT || 3000;

app.get("/health", (req, res) => res.json({ status: "ok" }));
app.get("/hello", (req, res) => res.json({ message: "Hello from Kubernetes" }));

app.listen(PORT, () => console.log(`API listening on port ${PORT}`));
```

Test it, then open `http://localhost:3000/hello` in a browser and stop with Ctrl+C:

```
node index.js
```

4. Add .gitignore and Dockerfile

.gitignore keeps `node_modules` and VS Code settings out of Git. The `Dockerfile` is the recipe for the image (the file has no extension).

.gitignore:

```
node_modules
.vscode
```

Dockerfile:

```
FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --omit=dev
COPY . .
EXPOSE 3000
CMD ["node", "index.js"]
```

Phase B: GitHub and automatic image build

5. Create the GitHub repo (browser)

New repository named `fhir-k8s-gitops`, **Public**, with no README, .gitignore or license. Public lets ArgoCD read it without credentials.

6. First commit and push

Sets a repo-local identity so this repo uses the `rprrr736` account, then pushes. A sign-in window may appear.

```
git init -b main
git config user.name "rprrr736"
git config user.email "<your rprrr736 email>"
git add .
git status
git commit -m "Initial commit: tiny Node API and Dockerfile"
git remote add origin https://rprrr736@github.com/rprrr736/fhir-k8s-gitops.git
git push -u origin main
```

7. Add the GitHub Actions workflow

On every push to `main` (except changes under `k8s/`), GitHub builds the `Dockerfile` and pushes the image to GHCR. No secrets needed, because the built-in `GITHUB_TOKEN` logs in.

```

mkdir .github\workflows
code .github\workflows\build.yml

name: Build and push image

on:
  push:
    branches: [main]
    paths-ignore:
      - "k8s/**"
  workflow_dispatch:

jobs:
  build:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write
    steps:
      - name: Check out code
        uses: actions/checkout@v4

      - name: Log in to GHCR
        uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${ github.actor }
          password: ${ secrets.GITHUB_TOKEN }

      - name: Build and push image
        uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: ghcr.io/rprrr736/fhir-k8s-gitops:v1

git add .
git commit -m "Add GitHub Actions workflow"
git push

```

Watch the **Actions** tab until the run is green.

8. Make the image public (browser)

New GHCR packages are private, so the cluster could not pull them. Profile → **Packages** → fhir-k8s-gitops → **Package settings** → **Danger Zone** → **Change visibility** → **Public**.

Phase C: Kubernetes YAML

9. Write the Deployment and Service

The Deployment says "run this image on port 3000 and check /health ". The Service gives it a stable address on node port 30080.

```
mkdir k8s
code k8s\deployment.yaml
code k8s\service.yaml
```

k8s/deployment.yaml:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: fhir-api
spec:
  replicas: 1
  selector:
    matchLabels:
      app: fhir-api
  template:
    metadata:
      labels:
        app: fhir-api
    spec:
      containers:
        - name: fhir-api
          image: ghcr.io/rprrr736/fhir-k8s-gitops:v1
          ports:
            - containerPort: 3000
          readinessProbe:
            httpGet:
              path: /health
              port: 3000
            initialDelaySeconds: 3
            periodSeconds: 5
```

k8s/service.yaml:

```
apiVersion: v1
kind: Service
metadata:
  name: fhir-api
spec:
  type: NodePort
  selector:
    app: fhir-api
  ports:
    - port: 80
      targetPort: 3000
      nodePort: 30080
```

```
git add .  
git commit -m "Add Kubernetes deployment and service"  
git push
```

Phase D: First deploy by hand (KillerCoda)

10. Open a Kubernetes playground

killercode.com → Playgrounds → **Kubernetes**. Sessions are temporary (about an hour), which is why everything lives in Git.

```
kubectl get nodes
```

11. Deploy with kubectl apply

Proves the image and YAML work before adding ArgoCD.

```
git clone https://github.com/rprrr736/fhir-k8s-gitops.git  
cd fhir-k8s-gitops  
kubectl apply -f k8s/  
kubectl get pods
```

Repeat get pods until Running and 1/1. ImagePullBackOff usually means the package is still private or the tag is wrong.

12. Call the API from the terminal

```
kubectl get svc fhir-api  
curl http://localhost:30080/health  
curl http://localhost:30080/hello
```

13. Call the API from Postman (optional)

Postman runs on your laptop, so it needs a public URL. In the KillerCoda page open **Traffic / Ports**, enter **30080**, and open it. Add the path yourself: <url>/hello. A bare URL shows "Cannot GET /", which only means your app answered but has no / route. Use a GET request in Postman, and keep the base URL in a variable, because each session gets a new one.

Phase E: ArgoCD

14. Install ArgoCD in the cluster

--server-side avoids an "annotation too long" error.

```
kubectl create namespace argocd  
kubectl apply -n argocd --server-side --force-conflicts -f https://raw.githubusercontent.com/argoproj/  
kubectl get pods -n argocd
```

Repeat until all seven pods are Running (1 to 3 minutes).

15. Write the ArgoCD Application (laptop)

One object that says "watch the k8s folder on main and deploy it to this cluster, automatically".

```
mkdir argocd
code argocd\application.yaml

apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: fhir-api
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/rprrr736/fhir-k8s-gitops.git
    targetRevision: main
    path: k8s
  destination:
    server: https://kubernetes.default.svc
    namespace: default
  syncPolicy:
    automated:
      prune: true
      selfHeal: true

git add .
git commit -m "Add ArgoCD application"
git push
```

16. Apply the Application in the cluster

```
cd ~/fhir-k8s-gitops
git pull
kubectl apply -f argocd/application.yaml
sleep 30
kubectl get applications -n argocd
```

You want Synced and Healthy. From now on ArgoCD deploys; you no longer run `kubectl apply -f k8s/`.

Phase F: Making changes through Git

17. Scale by changing Git

In `k8s/deployment.yaml` change `replicas: 1` to `replicas: 2`, save, then:

```
Select-String -Path k8s\deployment.yaml -Pattern "replicas"
git add .
git commit -m "Scale to 2 replicas"
git push
```

```
kubectl get pods
```

A second pod appears with no `kubectl` from you (ArgoCD polls about every 3 minutes).

18. The release loop, manual version (replaced by Phase J, step 35)

This is how releases worked before the tag edit was automated. Replace `vN` with the new tag (v2, v3 ...).

Never reuse a tag, because Kubernetes caches images by tag.

1. Change the code in `index.js`, and change the last line of `build.yml` to `...:vN`. Save both, then verify they are really saved:

```
Select-String -Path index.js -Pattern "message"
Select-String -Path .github\workflows\build.yml -Pattern "tags"
git status
```

2. Commit and push, then wait for a **green** run in the Actions tab:

```
git add .
git commit -m "Build vN image"
git push
```

3. Only after it is green, change the image line in `k8s\deployment.yaml` to `:vN`, then verify, commit and push:

```
Select-String -Path k8s\deployment.yaml -Pattern "image:"
git add .
git commit -m "Deploy vN image"
git push
```

4. In KillerCoda, ask ArgoCD to check now instead of waiting, then verify:

```
kubectl annotate application fhir-api -n argocd argocd.argoproj.io/refresh=hard --overwrite
sleep 30
kubectl get pods
kubectl get pods -o jsonpath='{.items[*].spec.containers[*].image}'
curl http://localhost:30080/hello
```

Phase G: FHIR routes

Use **fake test data only**. HAPI is a shared public server that anyone can read and change. Only delete patients you created.

19. Check the cluster can reach HAPI

```
curl -s -m 30 -H "Accept: application/fhir+json" "https://hapi.fhir.org/baseR4/Patient?_count=1" | he
```

You should see the start of a JSON Bundle .

20. Add the read routes

GET /patients (searches, optional ?name=) and GET /patients/:id . They call HAPI with Node's built-in fetch and return a trimmed patient (id , name , gender , birthDate). Locally, --use-system-ca is needed on the office network:

```
node --use-system-ca index.js
```

Open <http://localhost:3000/patients> , then <http://localhost:3000/patients/<id>> .

21. Add the write routes

POST /patients (create), PUT /patients/:id (replace the whole patient, so send all four fields) and DELETE /patients/:id . The complete final index.js is in the repo. It also uses app.use(express.json()) so request bodies are read.

22. Test locally in Postman

Start the app, then use these requests against <http://localhost:3000> in this order:

Request	URL	Expect
POST	/patients	201 and a new id
GET	/patients/<id>	200, same patient
PUT	/patients/<id>	200, changed name
DELETE	/patients/<id>	200 {"deleted": "<id>"}
GET	/patients/<id>	an error such as 404 or 410

For POST and PUT use Body → raw → JSON:

```
{
  "given": "Test",
  "family": "Demo",
  "gender": "female",
  "birthDate": "1990-01-01"
}
```

23. Ship the routes with the release loop

Run step 18 with v5 . (/hello was changed to say v5, so you can see the rollout.)

24. Rebuild the cluster from Git

When the playground expires, nothing is lost. In a new Kubernetes playground:

```
git clone https://github.com/rprrr736/fhir-k8s-gitops.git
cd fhir-k8s-gitops
kubectl create namespace argocd
kubectl apply -n argocd --server-side --force-conflicts -f https://raw.githubusercontent.com/argoproj/argoproj/master/examples/argocd.yaml
kubectl get pods -n argocd
```

When all seven pods are Running:

```
kubectl apply -f argocd/application.yaml
sleep 30
kubectl get applications -n argocd
kubectl get pods
```

ArgoCD creates the Deployment and Service from Git, with no `kubectl apply -f k8s/`.

25. Test the pod from Postman

Open **Traffic / Ports** for port **30080** to get the new session's URL, set it as the base URL, and repeat the table from step 22 against it (**POST** , **GET** , **PUT** , **DELETE** on `/patients`). The request path is Postman → KillerCoda URL → your pod → HAPI.

Phase I: ArgoCD web UI and self-healing

The UI shows the same things as `kubectl get applications` and `kubectl get pods`, as a picture. The setup is temporary and has to be repeated after every KillerCoda reset (steps 26 to 28 can be run together).

26. Make the ArgoCD server reachable

The ArgoCD server runs inside the cluster and is not exposed. This changes its Service to a NodePort, with port 30090 for plain HTTP and 30091 for HTTPS.

```
kubectl get pods -n argocd
kubectl patch svc argocd-server -n argocd -p '{"spec":{"type":"NodePort","ports":[{"name":"http","port":30090,"targetPort":8080}],"nodePorts":[{"name":"http","port":30090}]}'}
kubectl get svc argocd-server -n argocd
```

You should see `80:30090/TCP,443:30091/TCP`.

27. Switch ArgoCD to plain HTTP

ArgoCD normally forces HTTPS with a self-signed certificate, which tends to break behind KillerCoda's public URL proxy. `server.insecure` is fine for a throwaway practice playground and wrong for anything real.

```
kubectl patch configmap argocd-cmd-params-cm -n argocd --type merge -p '{"data":{"server.insecure":"true"}}'
kubectl rollout restart deployment argocd-server -n argocd
```

```
kubectl rollout restart deployment argocd-server -n argocd
kubectl rollout status deployment argocd-server -n argocd
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:30090
```

The last command should print `200`. A `301` or `308` means it is still redirecting to HTTPS, and a connection error usually means the pod is not ready yet.

28. Open the UI and log in

Print the admin password (it is random, and only valid for this playground):

```
kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" | base64 -d;
```

Then open KillerCoda's **Traffic / Ports**, enter **30090**, and open it. Log in with username **admin** and that password. You should see the dashboard with a `fhir-api` tile.

29. Read the application tree

Click the `fhir-api` tile. Follow the diagram from left to right: the Application, then the Service and Deployment, then a ReplicaSet, then the pods. The top shows sync status (`Synced` or `OutOfSync`) and health (`Healthy`, `Progressing`, `Degraded`). Click a pod and open its **Logs** tab to see API listening on port `3000`.

30. Watch a Git change reach the cluster

Keep the ArgoCD tab on the tree. On your laptop, change `replicas: 2` to `replicas: 3` in `k8s\deployment.yaml`, save, and verify before committing:

```
Select-String -Path k8s\deployment.yaml -Pattern "replicas"
git status
git add .
git commit -m "Scale to 3 replicas"
git push
```

In ArgoCD click **Refresh** (otherwise it finds the change within about 3 minutes). The status flips briefly to **OutOfSync**, then **Synced**, and a third pod box appears and goes from a yellow progressing icon to a green heart.

31. Read the deployment history

On the application page, open **History and Rollback** (labeled **Sync History** in some versions). Each entry shows a Git commit ID and message, such as `Scale to 3 replicas` or `Deploy v5 image`, so you get a visual trail of every release. **App Diff** shows what differs between Git and the cluster, and says there are no differences when everything is in sync. Rollback may be blocked while auto-sync is on, which is normal.

32. Self-heal test 1: change the Deployment by hand

This changes the Deployment spec, so ArgoCD (`selfHeal: true`) puts it back to match Git.

```
kubectl get deployment fhir-api
kubectl scale deployment fhir-api --replicas=1
kubectl get pods -w
```

Leave the watch running for about 30 to 40 seconds, then press `Ctrl+C` and confirm:

```
kubectl get deployment fhir-api
```

Pods go to `Terminating`, new ones appear, and the Deployment returns to `3/3` with no action from you. It happens quickly, so the **OutOfSync** flash in the UI is easy to miss, and you can tell it healed because some pod names are new and young.

33. Self-heal test 2: delete one pod

This changes nothing in the Deployment spec, so **Kubernetes** (the ReplicaSet) replaces the pod and ArgoCD does nothing.

```
kubectl get pods
kubectl delete $(kubectl get pods -o name | head -1)
kubectl get pods
```

Do not put `pod` in front of `$(...)`, because `-o name` already returns `pod/<name>`. The list still shows 3 pods, with one brand new.

Test	What you changed	Who fixed it	Why
<code>kubectl scale ... --replicas=1</code>	The Deployment spec	ArgoCD (self-heal)	The spec no longer matched Git
<code>kubectl delete pod ...</code>	One pod	Kubernetes (ReplicaSet)	Fewer pods than the spec asks for

Kubernetes keeps the pods matching the Deployment, and ArgoCD keeps the Deployment and the other YAML matching Git.

Phase J: Automatic tag update (the release is just a code push)

Until now you typed the image tag twice, in `build.yml` and in `deployment.yml`. Now the workflow builds the image with the **short commit ID** as its tag, then edits `k8s/deployment.yml` and commits it back to `main` itself. ArgoCD sees that commit and deploys. The tag is unique every time, so it can never be reused or forgotten, and the manifest is only changed after the image has been pushed, so a pod can't ask for an image that doesn't exist yet.

34. Replace the workflow with the automated one

Open the file with `code .github\workflows\build.yml`, delete everything, paste the following, and save. Paste it as-is, because YAML depends on exact spacing, and check that `if`, `else` and `fi` are on separate lines.

```
name: Build, push and update deployment
```

```
on:
  push:
    branches: [main]
    paths-ignore:
      - "k8s/**"
      - "argocd/**"
  workflow_dispatch:

jobs:
  build:
    runs-on: ubuntu-latest
    permissions:
      contents: write
      packages: write
    steps:
      - name: Check out code
        uses: actions/checkout@v4

      - name: Set image tag from commit
        id: vars
        run: echo "tag=${GITHUB_SHA::7}" >> "$GITHUB_OUTPUT"

      - name: Log in to GHCR
        uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${GITHUB_ACTOR}
          password: ${GITHUB_TOKEN}

      - name: Build and push image
        uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: ghcr.io/rprrr736/fhir-k8s-gitops:${GITHUB_OUTPUT.tag}

      - name: Update image tag in deployment.yaml
        run: |
          sed -i "s|image: ghcr.io/rprrr736/fhir-k8s-gitops:.*|image: ghcr.io/rprrr736/fhir-k8s-gitops:${GITHUB_OUTPUT.tag}|" deployment.yaml

      - name: Commit and push the change
        run: |
          git config user.name "github-actions[bot]"
          git config user.email "41898282+github-actions[bot]@users.noreply.github.com"
          git add k8s/deployment.yaml
          if git diff --cached --quiet; then
            echo "No change to commit"
          else
            git commit -m "Deploy ${GITHUB_OUTPUT.tag}"
            git pull --rebase origin main
            git push origin main
          fi
```

Verify it is saved, then commit **only that file** (`git add .` would also pick up any stray untracked files in the repo):

```
Select-String -Path .github\workflows\build.yml -Pattern "contents: write","GITHUB_SHA","sed -i","git
git status
git add .github/workflows/build.yml
git commit -m "Automate image tag update in deployment.yaml"
git push
```

The `permissions: contents: write` line is what lets the workflow push the manifest change. Because `k8s/**` and `argocd/**` are in `paths-ignore`, the bot's commit does not start another build, so there is no loop.

35. The new release loop

1. Change the code, verify it is saved, and commit only the files you changed:

```
Select-String -Path index.js -Pattern "message"
git status
git add index.js
git commit -m "Change hello message"
git push
```

2. Wait for a **green** run in the Actions tab. It builds the image, pushes it, and commits `Deploy <id>` to `k8s/deployment.yaml`.

3. **Pull the bot's commit before your next change.** The bot added a commit to GitHub that your laptop doesn't have, and without this the next `git push` is rejected with "fetch first":

```
git pull
git log --oneline -3
Select-String -Path k8s\deployment.yaml -Pattern "image:"
```

The log shows a `Deploy <id>` commit on top, and the image line ends in the same 7-character id.

36. Verify in the cluster

ArgoCD finds the new commit by itself within about 3 minutes. The refresh annotation is optional and just makes it check now.

```
kubectl annotate application fhir-api -n argocd argocd.argoproj.io/refresh=hard --overwrite
sleep 30
kubectl get pods
kubectl get pods -o jsonpath='{.items[*].spec.containers[*].image}'
curl http://localhost:30080/hello
```

You should see new pods starting while the old ones show `Terminating` (a rolling update, so there is no downtime), the new image tag, and the changed message. The `jsonpath` line can list both the old and new images for a few seconds, because terminating pods are still on the list.

Lessons learned

- **Check saves before committing.** Run `Select-String` and `git status`. An unsaved edit once produced a `v2` image with old code.
- **Never reuse an image tag.** Commit-ID tags (Phase J) make this automatic. With the manual loop, use a new tag for every change.
- **Build first, deploy second.** The automated workflow only edits `deployment.yaml` after the image is pushed.
- **Two triggers:** a code push triggers the build, and a YAML push triggers the deploy. `paths-ignore` for `k8s/**` and `argocd/**` stops those edits from rebuilding, and the bot's own commit does not start another run.
- **`git pull` before every push (Phase J).** The bot commits to `main`, so your laptop is behind after each release, and a push without a pull is rejected with "fetch first".
- **Add files by name** (`git add index.js`) rather than `git add .` when the repo folder has untracked files you don't want in a public repo.
- **ArgoCD polls about every 3 minutes.** Use the `refresh=hard` annotation to check right away.
- **"Cannot GET /"** means your app answered, and the URL just needs `/hello` or `/patients`.
- **Docker Hub only stores images.** Something with Docker still has to build them, and here GitHub Actions does it.
- **`apiVersion` in YAML is the Kubernetes API version.** It is unrelated to your image tag.

Ideas for later

- Jenkins in place of GitHub Actions for the build (needs somewhere to run, which is hard on an office laptop).
- Add a React page, SMART/OAuth login, or the AI agent and RAG on top.